

45 COMPONENTS · 7 FUNCTION FAMILIES

Lattice Component Gallery

Every `example.md` in lib/components/, rendered.

SECTION 01 · 4 COMPONENTS

Anchor

Function · Form · Substance · Finish.

docs/design-system.md

SECTION 03

Inventory

ANCHOR FAMILY · LIGHT DIVIDER

Subtopic introduces a specific topic within a section.

LATTICE · COMPONENT GALLERY

The component model in one deck

Forty-five components, one page each, by function family.

SECTION 02 · 4 COMPONENTS

Statement

AUDIENCE RECALL

92%

of the audience remembers a single number from a deck.
Make it count.

The catch-all prose slide.

Use *CONTENT* when no more specific component fits. A heading, a paragraph, optionally a short bullet list. Keep the slide under forty words.

For lists of items, prefer `list`, `cards-grid`, or `cards-stack`.

For comparisons, prefer `compare-prose` or `compare-table`.

For single metrics, prefer `big-number` or `stats`.

“

The signal was always there. We just didn't have a system that forced us to look at it before we'd already decided. The framework's value isn't the data — it's the moment in the calendar when the question gets asked.

”

— Head of Product, Pilot Team 3



A featured statement on the left lands the argument.

Featured panel + list • split-panel

USE WHEN

One claim deserves emphasis.

The panel headline carries the weight; the right side substantiates.

You need both a thesis and proof.

Statement on the left, evidence list on the right.

The right side is short.

Three to four supporting points; longer lists belong in

`cards-stack`

.

Modifier mirror flips it.

Image-style decks often want the panel on the right.

SECTION 03 · 12 COMPONENTS

Inventory

Who owns each part of the lifecycle.

Author.	Drafts the deck; owns content and framing.
Reviewer.	Validates clarity, factual accuracy, and audience-fit.
Engineer.	Ensures the build path renders the same PDF Marp preview shows.
Designer.	Owens the visual contract; palette tokens, layout balance, typography.
Operator.	Schedules the briefing; controls the room and the projector.

What this deck covers.

01 The four-layer model — Function · Form · Substance · Finish

02 Component manifests — the single source of truth

03 The forty-five shipped components, grouped by function

04 Discovery — scaffolder, snippets, this gallery

05 What ships next — open questions and follow-ups

When to reach for cards-grid.

Parallel items

Four cards or fewer, each item gets equal weight in the layout.
Audience compares them at a glance.

Scannable at a glance

The audience absorbs the whole set in one look — no scrolling, no eye-leaping between rows.

Equal information density

Each card carries roughly the same text length. Uneven density makes the grid feel unbalanced.

Order is decorative

When sequence carries meaning, use list-steps or list-criteria instead.
cards-grid is for parallel options.

Two cards, equal weight, side-by-side.

Use for an explicit pair

Two options, two phases, two artifacts presented with equal weight.
The slide reads as a comparison without taking sides.

Different from compare-prose

compare-prose adds connector chrome and a chosen modifier; cards-side stays neutral. Pick cards-side when neither option is the winner.

When to reach for cards-stack.

Vertical reading order matters. The audience scans top to bottom, not grid-style. Each card builds on the previous one as the eye moves down the slide.

Each card has more body than a grid card. Two sentences instead of one. cards-grid forces parallel density; cards-stack lets each card breathe with longer body text.

Two to three items, not four-plus. Beyond three cards the slide overflows. For more items, split across multiple slides or switch to cards-grid with shorter text.

When the items want full-width rows.

1 Each item has substantial body text

One to two sentences per item, more than a cards-grid card can hold without crowding. The row layout gives the body room to breathe.

2 The slide scans top-to-bottom

Reading order is sequential rather than parallel. The audience absorbs one row at a time rather than the whole set at a glance.

3 Three or four rows feels right

Beyond four rows the slide gets dense. For more items prefer list-tabular; for fewer items with shorter body prefer cards-stack.

Pre-flight checklist for a new component.

- ☒ Pick function and form coordinates per the spec
- ☒ Write the manifest with name, function, form, substance, and slots
- ☒ Author CSS rules scoped to the section class
- ☐ Add a transform module if substance is structure or series
- ☐ Write a substantive example and README
- ☐ Update the templates catalog reference
- ☐ Add unit tests under the new component test path

Glossary

C - S

TERM	DEFINITION
Component	A self-contained unit at lib/components, one folder per component, with manifest plus styles plus example plus optional transform plus README.
Function	The communication purpose a slide serves; one of seven families (Anchor, Statement, Inventory, Comparison, Progression, Evidence, Imagery).
Form	The spatial composition of a slide; one of eleven shapes (bookend, divider, canvas, grid, stack, ledger, panel, matrix, scatter, timeline, split).
Manifest	The JSON description of a component, consumed by the scaffolder, snippets, docs catalog, and autocomplete.
Substance	The kind of data that fills the form; one of four (prose, structure, series, graph).

When the items truly are a list.

Five to six short points, each under twelve words.

No internal structure per item — if items have title + body, use cards-stack instead.

Numbered (ol) when order matters; bulleted (ul) when it does not.

Inline-code metadata at the end of a row becomes a pill via the universal-pill recipe.

For richer items with descriptions, prefer list-tabular.

The four substance contracts a component plugs into.

01	Prose	Headings, paragraphs, inline emphasis — Marp markdown into semantic HTML.	CSS-ONLY; NO POST-PROCESSOR REQUIRED
02	Structure	Headings plus nested lists with conventions; a post-processor rewrites the list into purpose-built DOM.	PER-COMPONENT TRANSFORM.JS IN LIB/COMPONENTS
03	Series	Tabular DSL — axes and datapoints as bullets, parsed into geometry.	CHART-FAMILY KERNEL (RADAR, QUADRANT, PIECHART, GANTT, KANBAN)
04	Graph	External graph language (Mermaid today; D2 or PlantUML in the future).	EXTERNAL CLI; PALETTE INJECTED AT BUILD TIME

How we make calls when the spec is silent.

- 01 Default to the choice that is cheaper to reverse.

- 02 Name the actor, never the system.

- 03 Write down the bet on the same slide as the choice.

- 04 Form follows function — let the audience need shape the layout.

- 05 One main idea per slide; split if you cannot summarise it in one sentence.

What this section will tell you, in five lines.

Components stay short — `cards-grid` not `inventory.grid.cards`.

The four layers organise the catalog; they do not name components.

Manifests are the single source of truth for every component.

Discovery happens via the scaffolder and IDE snippets, not the directive.

Forty-five components ship — one folder each.

SECTION 04 · 7 COMPONENTS

Comparison

What the manifest refactor produced.

BEFORE.

35 layouts scattered across one 10,382-line lattice.css monolith. Per-layout rules grepped, not folder-located. No central metadata.



AFTER.

45 components self-contained at lib/components, one folder each with manifest plus styles plus example plus README. Bundler concatenates per-component CSS; loader exposes the catalog via JSON.

BEFORE & AFTER · COMPONENT MANIFEST LOADING

Flat-file lookup versus folder-shape lookup.

BEFORE · FLAT FILE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    `${name}.json`
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

AFTER · FOLDER SHAPE

```
const fs = require('node:fs');
const path = require('node:path');

function loadOne(name) {
  const p = path.join(
    __dirname, 'lib', 'components',
    name, 'manifest.json'
  );
  return JSON.parse(fs.readFileSync(p, 'utf8'));
}

const cards = loadOne('cards-grid');
```

Two options, equal weight, head-to-head.

FIRST OPTION

Two-sentence description of the first option, including the strongest argument for it. Equal-density prose lets the audience compare line by line.



SECOND OPTION

Two-sentence description of the second option, including the strongest argument for it. Modifier chosen marks the verdict when a decision has been made.

Where the four substance contracts come from.

SUBSTANCE	AUTHOR WRITES	RENDERER	OUTPUT
prose	headings, paragraphs, lists	Marp markdown → semantic HTML	DOM
structure	nested lists with conventions	lib/*.js post-processor	DOM
series	tabular DSL (axes + datapoints)	chart-family kernel	SVG
graph	external graph language	external CLI (mmdc, future d2)	SVG

What we are doing.

CHOSEN PATH.

Self-contained per-component folders at lib/components, one folder per component. Holds manifest plus styles plus optional transform plus example plus README.

REJECTED OPTION.

Flat files alongside each other in lib/components. Defeats the self-contained goal and leaves transform.js scattered.

Where each component substance lives.

Author-driven · DOM output.

prose — headings + paragraphs
structure — nested lists

Author-driven · SVG output.

series — tabular DSL
graph — external language

Data-driven · DOM output.

(Lattice does not target this cell)

Data-driven · SVG output.

chart-family kernels — radar, quadrant, piechart, gantt, kanban,
progress, timeline-list

Which option meets the criteria.

Folder shape.

- ☒ Self-contained per component
- ☒ Familiar pattern from other libraries
- ☒ Tests can live with their component

Flat files.

- ☒ Less restructuring upfront
- ☐ Per-component grouping by filename only
- ☐ No room for transform.js or example.md

Hybrid.



- ☐ Manifest stays flat, other files in subfolder
- ☐ Splits the component across two locations
- ☐ Defeats the self-contained goal

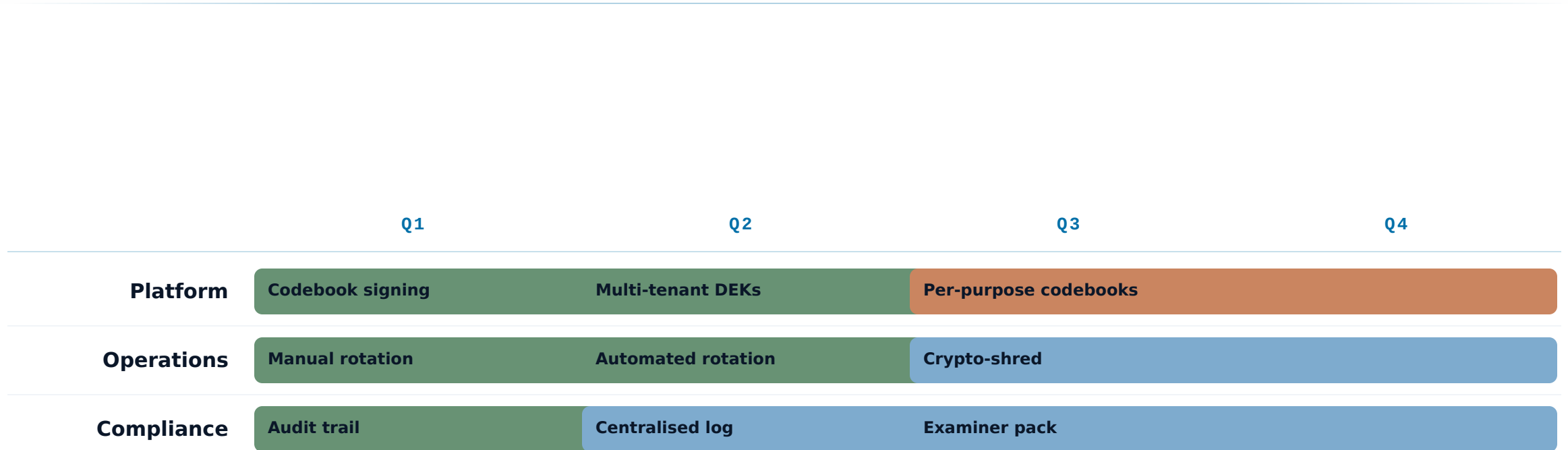
SECTION 05 · 6 COMPONENTS

Progression

2026 Q1 → 2026 Q4

What ships in each phase, by workstream.

Three workstreams across four quarters. Status pills tint each bar.



Where Phase 2 work stands today.

Four columns, mixed card density. Size badge sits in the title row.

BACKLOG

Per-purpose codebooks S

Crypto-shred runbook M

Dependency dashboard S

IN PROGRESS

Multi-tenant DEKs M

platform

at-risk

Examiner pack v2 L

compliance

REVIEW

Centralised log S

compliance

DONE

Codebook signing M

Manual rotation S

What every component manifest must satisfy.

01**Stable name**

Kebab-case, matching the class directive authors type when invoking the component.

02**Function coordinate**

One of seven families per the four-layer model: anchor, statement, inventory, comparison, progression, evidence, imagery.

03**Form coordinate**

One of eleven spatial shapes: bookend, divider, canvas, grid, stack, ledger, panel, matrix, scatter, timeline, split.

04**Substance coordinate**

One of four plugin contracts: prose, structure, series, graph.

05**Skeleton plus example**

Skeleton scaffolds blank slides for the new-slide CLI; example.md demonstrates the component substantively for the gallery.

How to add a new component to Lattice.

STEP 01

Create the component folder with a manifest declaring name, function, form, substance, slots, and skeleton.

STEP 02

Write the styles scoped to the section class. Wrap in the components layer once the cascade migration completes.

STEP 03

Add a transform module if the substance is structure or series. Wire it into all three render paths.

STEP 04

Author example.md and README.md. Regenerate the catalog gallery from the manifests.

STEP 05

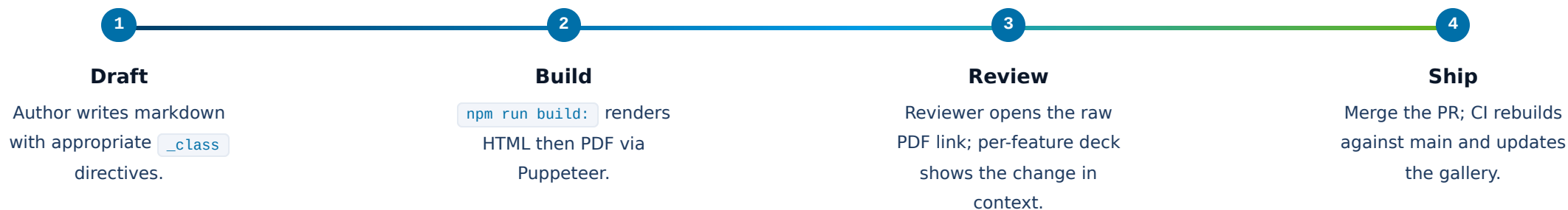
Add a unit test under the component test path. Run the full suite locally before pushing.

What ships in each phase, by workstream.

WORKSTREAM	FOUNDATION	HARDENING	SCALE
	Q2 2026	Q3 2026	Q4 2026
Platform	✓ Codebook signing	🛩 Multi-tenant DEKs	○ Per-purpose codebooks
Operations	✓ Manual rotation	🛩 Automated rotation	○ Crypto-shred
Compliance	✓ Audit trail	✓ Centralised log	○ Examiner pack
SDK	✓ Java	🛩 .NET	○ Polyglot parity

State markers `[x]/[-]/[ ]/[/]` are universal: ✓ shipped, 🛩 in flight, ○ planned, / out of scope.

How a deck moves from draft to share.



SECTION 06 · 10 COMPONENTS

Evidence

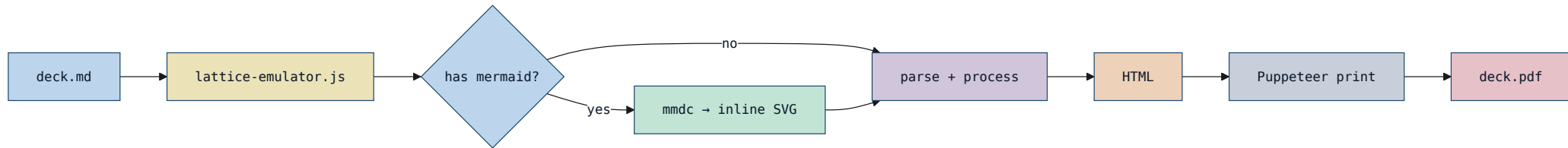
What loading a manifest looks like.

```
const { loadAll, groupByFunction } = require("../lib/components");

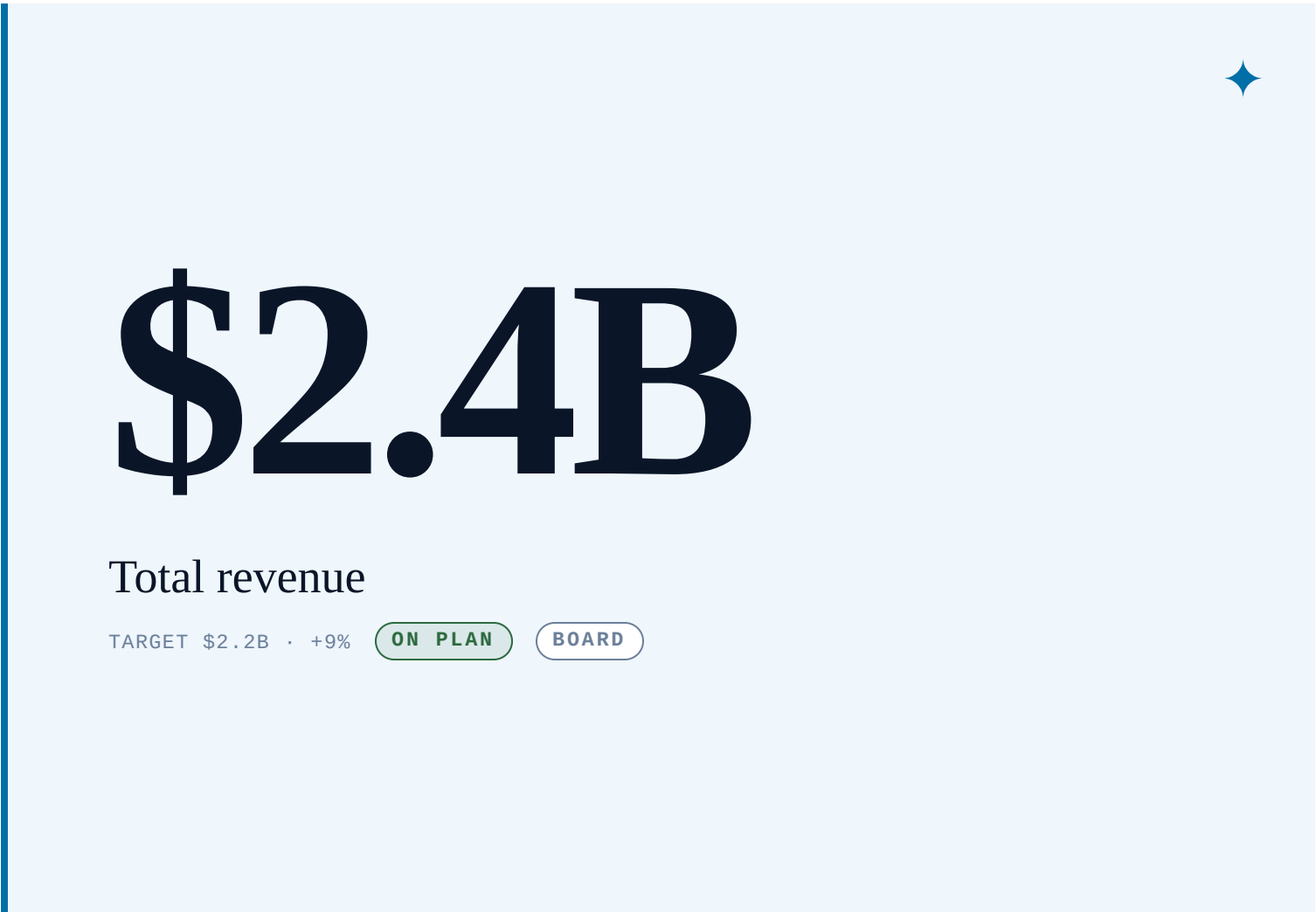
const manifests = loadAll();           // 45 components, validated
const byFunction = groupByFunction(manifests);

for (const m of byFunction.evidence) {
  console.log(m.name, m.form, m.substance);
}
```

How a Lattice slide goes from markdown to PDF.



Revenue ahead of plan; margin and cash both expanded.



42%

Gross margin

+2PP QOQ ON PLAN AUDIT

\$1.1B

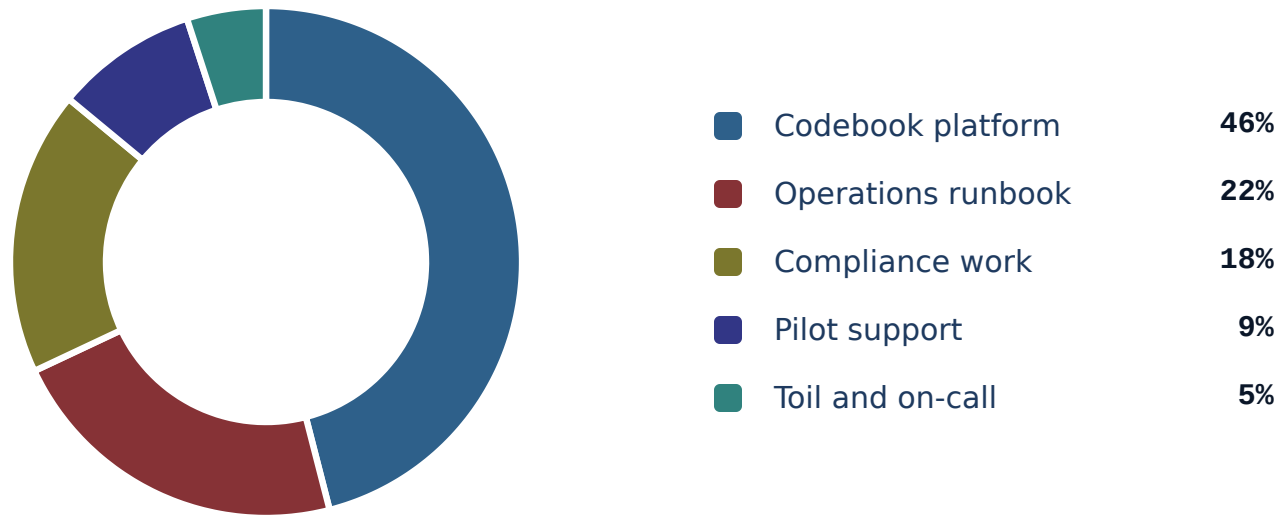
Cash & equivalents

+\$180M QOQ ON PLAN INVESTOR

H1 2026 · 1,840 PERSON-HOURS

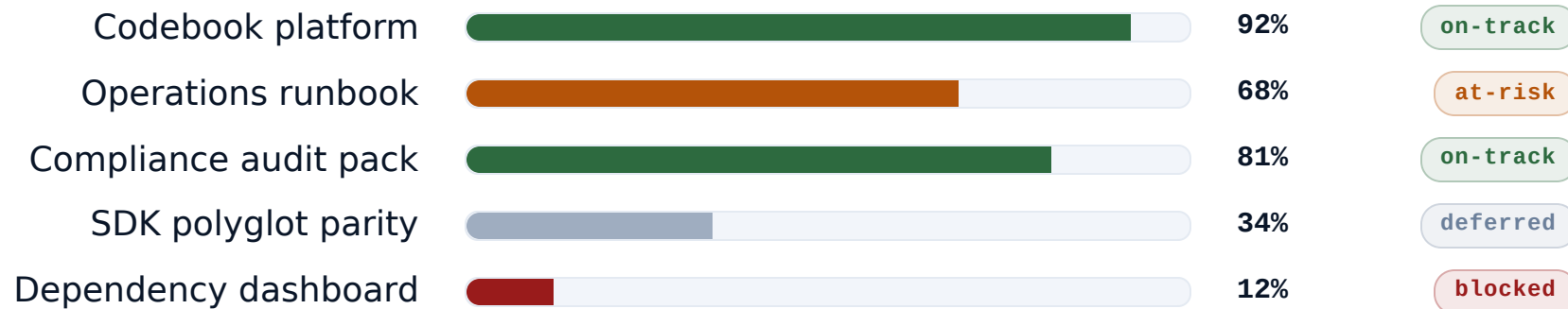
Where the engineering quarter went.

Wedges drawn proportionally; legend reads in author order with raw values.



Phase 1 readiness, by workstream.

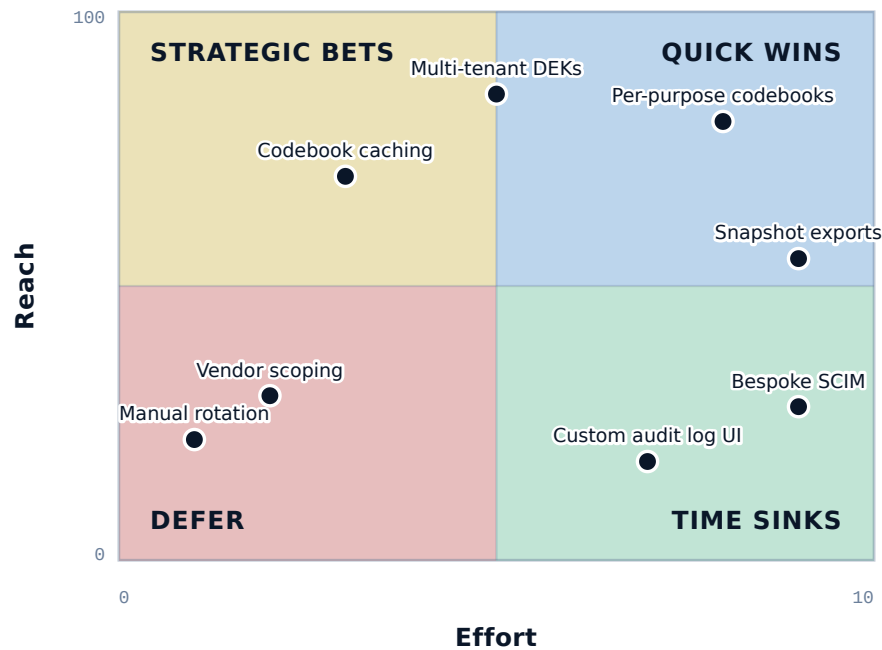
Snapshot taken at 14:00 UTC. Status pills tint the bar fill.



EFFORT 0-10 → REACH 0-100

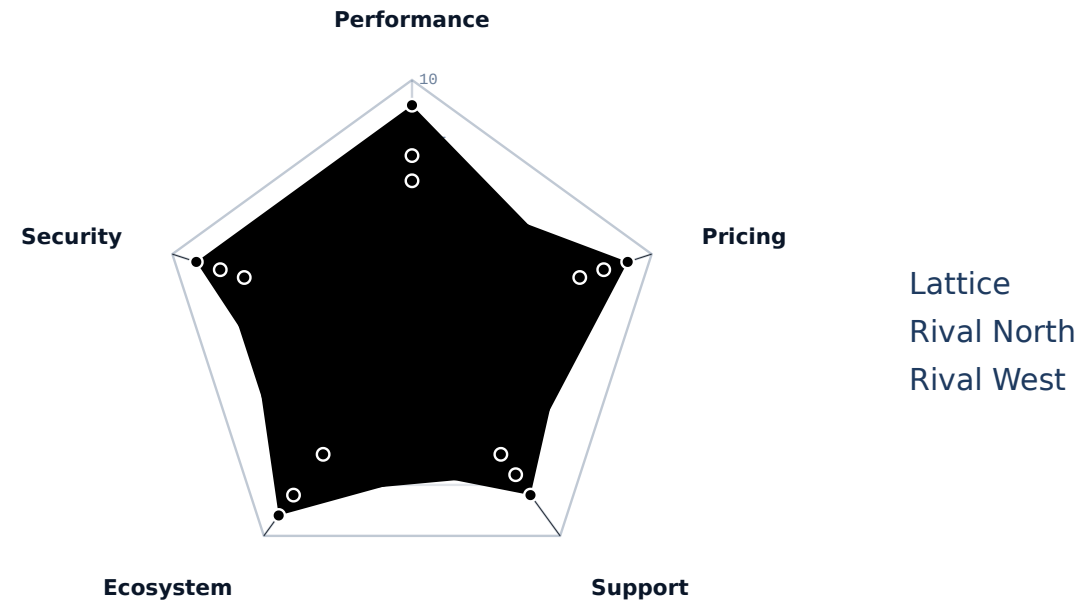
Where to put the next dollar.

Effort estimated in story-points; reach as percent of addressable users.



SCALE · 0-10

How we stack up across the buying criteria.



IMPACT · PILOT RESULTS

Six months of results across four product teams.

Measured against pre-framework baseline, same teams, same market conditions.

73%

FASTER CLOSE

4.2×

SIGNAL RECALL

\$1.2M

PREVENTED LOSSES

−18d

AVG CYCLE TIME

How the codebook architecture arrived in production.

Four stages over eighteen months. Date pill leads each item; status pill trails.



What this branch named, by weight.

folder manifest variant
gallery component form
substance function

SECTION 07 · 2 COMPONENTS

Imagery

Applying the criteria, here is where the evidence points.

Self-contained component folders.



One folder per component holding manifest, styles, transform (if needed), example, and README. Matches the pattern every mature design system uses.

Bundler concatenates CSS at build time.

Per-component sources combine into the shipped lattice stylesheet via the build-css tool. Committed bundle with a CI gate.

Manifests are the single source of truth.

Scaffolder, snippets, this gallery, and docs all read from the same JSON.

Tests stay scoped.

One test file per component under the components test path, runnable as a scoped npm script.

Image right is the default — text leads, evidence follows.

The image fills its half-canvas slot edge-to-edge. A 1px hairline marks the join between text and image — boardroom polish, no placeholder pattern visible behind a real photo. Replace the bg image directive with your own asset.



That is every component.

docs/design-system.md · lib/components/